



# Web services

---

Prof. Alessandro Brawerman



# {JSON} Placeholder

---

- Este projeto usa a API

<https://jsonplaceholder.typicode.com/>

- para fazermos um projeto completo com get, post, put e delete.
- Iremos passar acessar cada um dos requests para selecionar um post, enviar um novo post, atualizar um post já existente e excluir um post.
- A URL acima informa como integrar com a API disponível e usar os endpoints necessários.

# {JSON} Placeholder

---

- Crie o projeto placeholderPost.
- Adicione as dependências do Retrofit no build.gradle do app.
- Adicione a permissão de Internet no arquivo manifesto.
- Crie o Model – Post.

```
data class Post(  
    val userId: Int,  
    val id: Int? = null, // O ID será gerado automaticamente no POST  
    val title: String,  
    val body: String  
)
```

# {JSON} Placeholder

---

- Crie a interface PostApi.
- Os métodos que criam ou atualizam um post, levam a informação para o Body do request.

```
interface PostApi {  
  
    // Requisição GET para obter um post específico  
    @GET("posts/{id}")  
    suspend fun getPost(@Path("id") postId: Int): Post  
  
    // Requisição POST para criar um novo post  
    @POST("posts")  
    suspend fun createPost(@Body newPost: Post): Post  
  
    // Requisição PUT para atualizar um post existente  
    @PUT("posts/{id}")  
    suspend fun updatePost(@Path("id") postId: Int,  
                           @Body updatedPost: Post): Post  
  
    // Requisição DELETE para deletar um post específico  
    @DELETE("posts/{id}")  
    suspend fun deletePost(@Path("id") postId: Int)  
}
```

# placeholderPost View

<LinearLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:padding="16dp">
```

<Button

```
    android:id="@+id/buttonGet"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="GET Post"  
    android:onClick="doIt"/>
```

<Button

```
    android:id="@+id/buttonPost"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="POST Criar Post"  
    android:onClick="doIt"/>
```

## placeholderPost View

```
<Button
```

```
    android:id="@+id/buttonPut"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="PUT Atualizar Post"  
    android:onClick="dolt"/>
```

```
<Button
```

```
    android:id="@+id/buttonDelete"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="DELETE Deletar Post"  
    android:onClick="dolt"/>
```

```
<TextView
```

```
    android:id="@+id/textViewResult"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:textSize="16sp"  
    android:text="Resultado aparecerá aqui"  
    android:paddingTop="16dp" />
```

```
</LinearLayout>
```

# Controller MainActivity

```
class MainActivity : AppCompatActivity() {

    private lateinit var textViewResult: TextView
    private lateinit var postApi: PostApi

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        textViewResult = findViewById(R.id.textViewResult)

        // Configura Retrofit
        val retrofit = Retrofit.Builder()
            .baseUrl("https://jsonplaceholder.typicode.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        postApi = retrofit.create(PostApi::class.java)
    }
}
```

# Controller MainActivity

```
fun doIt(view: View) {  
    when(view.id) {  
        R.id.buttonGet -> getPost((1 ≤ .. ≤ 100).random())  
        R.id.buttonPost -> createPost(Post(userId = 1, title = "Novo Post",  
            body = "Este é um novo post."))  
        R.id.buttonPut -> updatePost(postId: 1, Post(userId = 1,  
            title = "Post Atualizado", body = "Este post foi atualizado."))  
        R.id.buttonDelete -> deletePost(postId: 1)  
    }  
}
```



# Controller MainActivity

```
// Função para buscar um post usando GET
private fun getPost(postId: Int) {
    lifecycleScope.launch {
        try {
            val post = withContext(Dispatchers.IO) {
                postApi.getPost(postId)
            }

            textViewResult.text = "ID: ${post.id}\n\n" +
                "Título: ${post.title}\n\n" +
                "Corpo: ${post.body}"

        } catch (e: Exception) {
            Log.e(tag: "MainActivity", msg: "Erro ao buscar o post", e)
            textViewResult.text = "Erro ao buscar o post."
        }
    }
}
```

# Controller MainActivity

```
// Função para criar um novo post usando POST
private fun createPost(newPost: Post) {
    lifecycleScope.launch {
        try {
            val createdPost = withContext(Dispatchers.IO) {
                postApi.createPost(newPost)
            }

            textViewResult.text = "Post criado com sucesso!\n\n" +
                "ID: ${createdPost.id}\n\n" +
                "Titulo: ${createdPost.title}\n\n" +
                "Corpo: ${createdPost.body}"

        } catch (e: Exception) {
            Log.e(tag: "MainActivity", msg: "Erro ao criar o post", e)
            textViewResult.text = "Erro ao criar o post."
        }
    }
}
```

# Controller MainActivity

```
// Função para atualizar um post usando PUT
private fun updatePost(postId: Int, updatedPost: Post) {
    lifecycleScope.launch {
        try {
            val updated = withContext(Dispatchers.IO) {
                postApi.updatePost(postId, updatedPost)
            }

            textViewResult.text = "Post atualizado com sucesso!\n\n" +
                "ID: ${updated.id}\n\n" +
                "Título: ${updated.title}\n\n" +
                "Corpo: ${updated.body}"

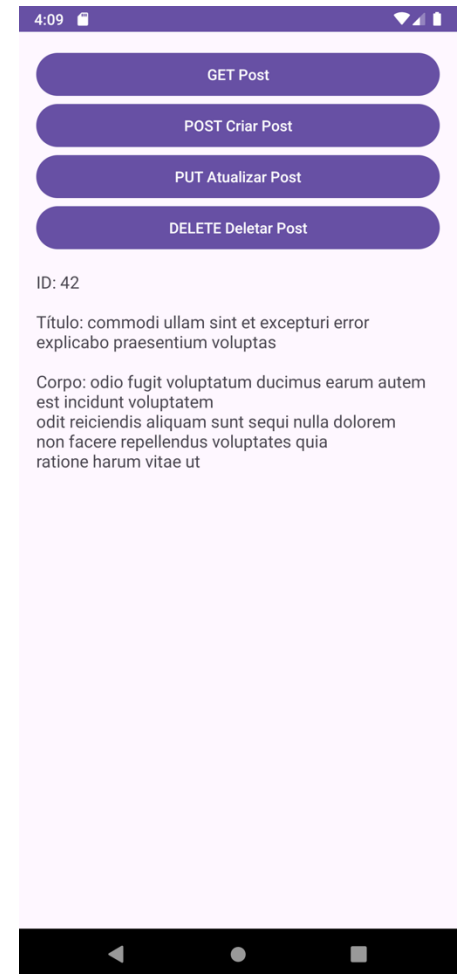
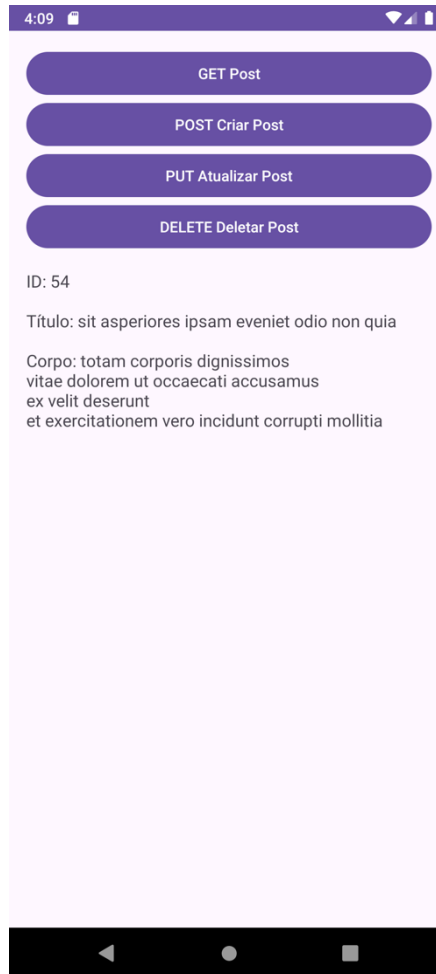
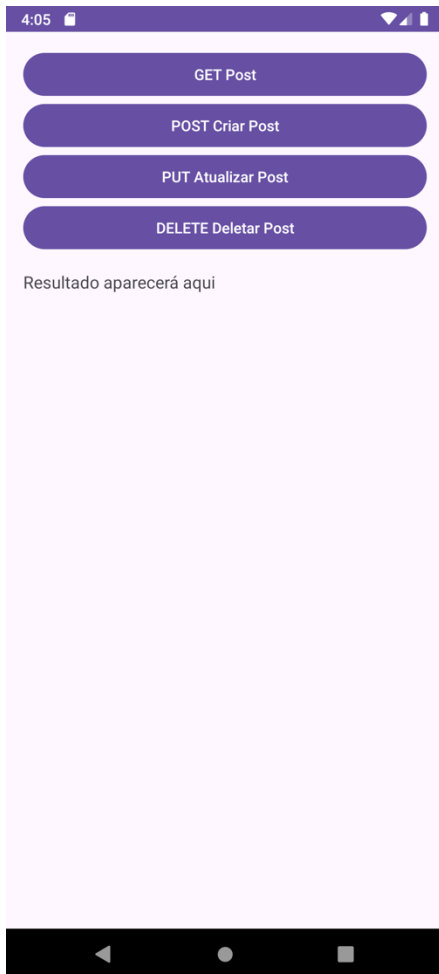
        } catch (e: Exception) {
            Log.e(tag: "MainActivity", msg: "Erro ao atualizar o post", e)
            textViewResult.text = "Erro ao atualizar o post."
        }
    }
}
```

# Controller MainActivity

```
// Função para deletar um post usando DELETE
private fun deletePost(postId: Int) {
    lifecycleScope.launch {
        try {
            withContext(Dispatchers.IO) {
                postApi.deletePost(postId)
            }

            textViewResult.text = "Post $postId deletado com sucesso!"
        } catch (e: Exception) {
            Log.e(tag: "MainActivity", msg: "Erro ao deletar o post", e)
            textViewResult.text = "Erro ao deletar o post."
        }
    }
}
```

# placeholderPost – Execute o APP



# placeholderPost – Execute o APP

---

